

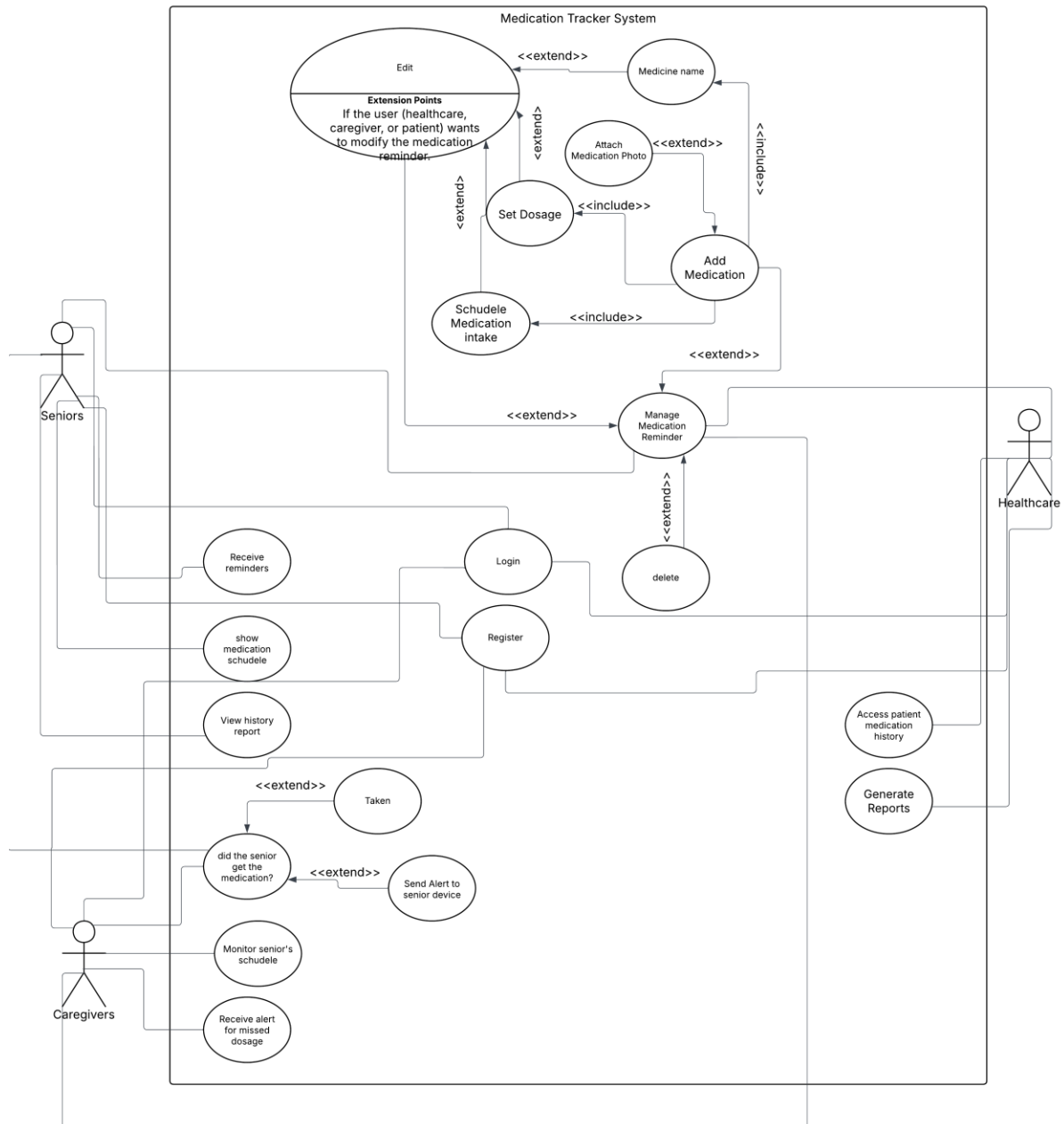


CPIT 251 - Project

(Medication Tracker)

Student Name	ID
1. Ali Salem Battiyah	2338097
2. Bandar al-Jedani	2237428
3. Ahmad Al-Ghamdi	2337548
4. Abdulaziz Al-Orabi	2339698
Section: IT2	

Use Case diagram



Selected Use Case

The system's core use case is **Manage Medication Reminder**, which allows users to add, edit, and delete medication reminders while recording key details such as dosage, schedule, medicine name, and an optional photo for easy identification.

This feature ensures that seniors never miss a dose while caregivers and healthcare providers stay informed in real time.

Use Case Description: Manage Medication Reminder

Field	Details
Use Case Name	Manage Medication Reminder
Actors	Senior, Caregiver, Healthcare Provider
Preconditions	User must be logged in
Postconditions	Reminder is created, updated, or deleted successfully
Includes	Set Dosage, Schedule Medication Intake, Attach Medication Photo, Medicine Name
Extends	Edit (if user modifies reminder), Delete (if user removes reminder)

Main Flow

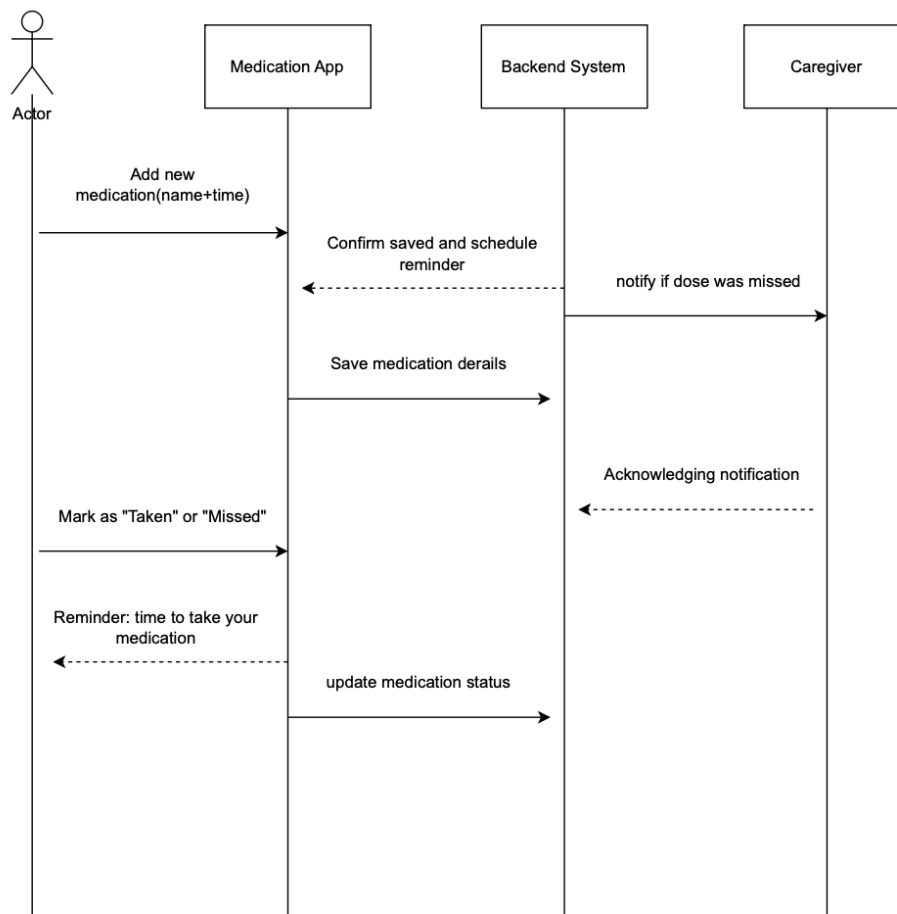
- User selects “Manage Medication Reminder.”
 - System displays existing reminders.
 - User chooses to Add, Edit, or Delete a reminder.
 - For Add, the user must enter name, dosage, schedule, and optional photo.
 - For Edit, the user may update reminder details.
 - For Delete, the user confirms removal.
 - System saves and updates all changes.
-

Alternative Flow

- If any field is missing or invalid, the system prompts the user to correct it.
 - If the device is offline, the system stores the reminder data locally and automatically synchronizes it once the internet connection is restored.
-

Sequence Diagram Description

The **Sequence Diagram** demonstrates the flow of interactions between the *Senior User*, the *Medication Tracker System*, and the *Database* during the **Manage Medication Reminder** process.



Step-by-Step Explanation

1. **Senior User** opens the application and selects *Manage Medication Reminder*.
2. The **System** requests reminder data from the **Database** and displays it to the user.

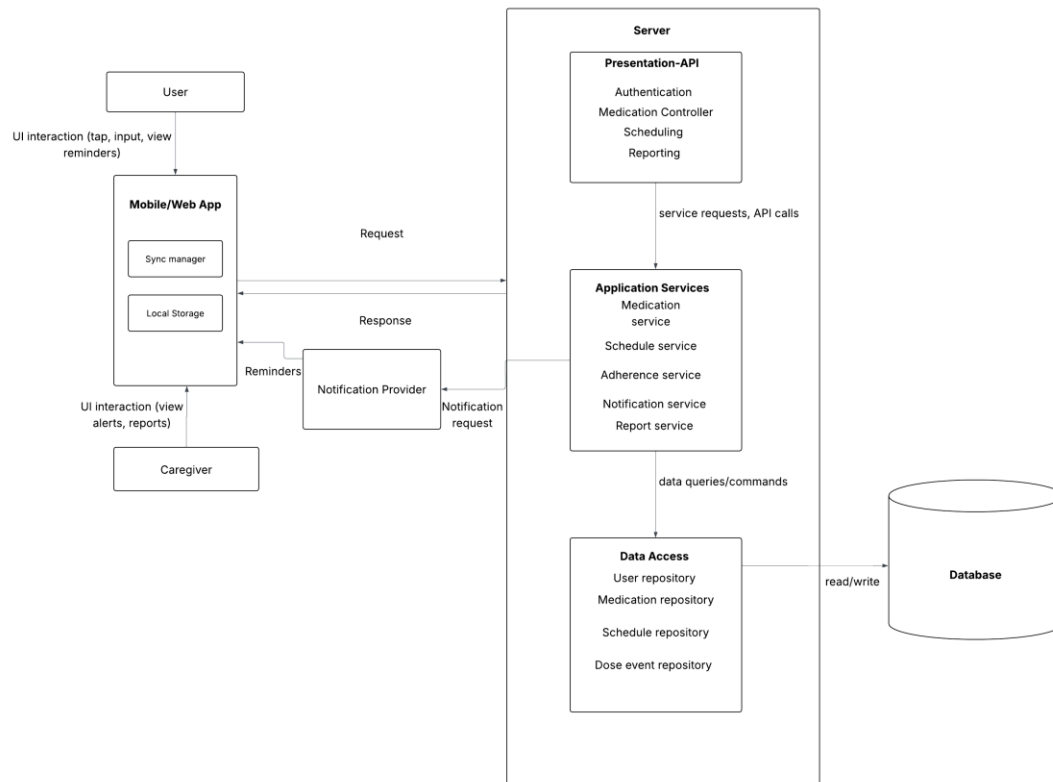
3. The **User** performs one of the following actions: Add, Edit, or Delete a reminder.
4. The **System** validates the input data (medicine name, dosage, schedule, photo).
5. If invalid, an error message is displayed.
6. The **System** saves valid data into the **Database**.
7. If there is **no internet connection**, the **OfflineSync** class stores the action locally in a queue.
8. Once connectivity is restored, **OfflineSync** automatically synchronizes the local data with the main **Database**.
9. The **System** confirms that the operation (Add/Edit/Delete) is successful.
10. A **Notification** is scheduled for the next dose, and if missed, the **Caregiver** is alerted.

Involved Components

- i. **Actors:** Senior User, Caregiver
 - ii. **System Entities:** ReminderManager, OfflineSync, Database, NotificationService
 - iii. **Goal:** To visualize how reminders are managed and synchronized both online and offline.
-

Medication Tracer Architecture

Our proposed system uses Client-Server architecture combined with a Layered design on the server side. **First**, the client (mobile or web application) handles user interaction, local data storage, and synchronization when internet connectivity is available. **Second**, the server is structured into three layers: API Layer-Application Services Layer-Data Access Layer. The API Layer provides web-based entry points and manages authentication, authorization, and data validation. The Application Services Layer implements the business logic of the system, including medication management, scheduling, adherence tracking, notifications, and reporting. The Data Access Layer is responsible for all interactions with the database, ensuring secure and efficient CRUD (create, read, update, delete) operations through repositories. This layered design promotes separation of concerns, maintainability, and security, while the client-server setup enables centralized data control and synchronization between offline and online modes.

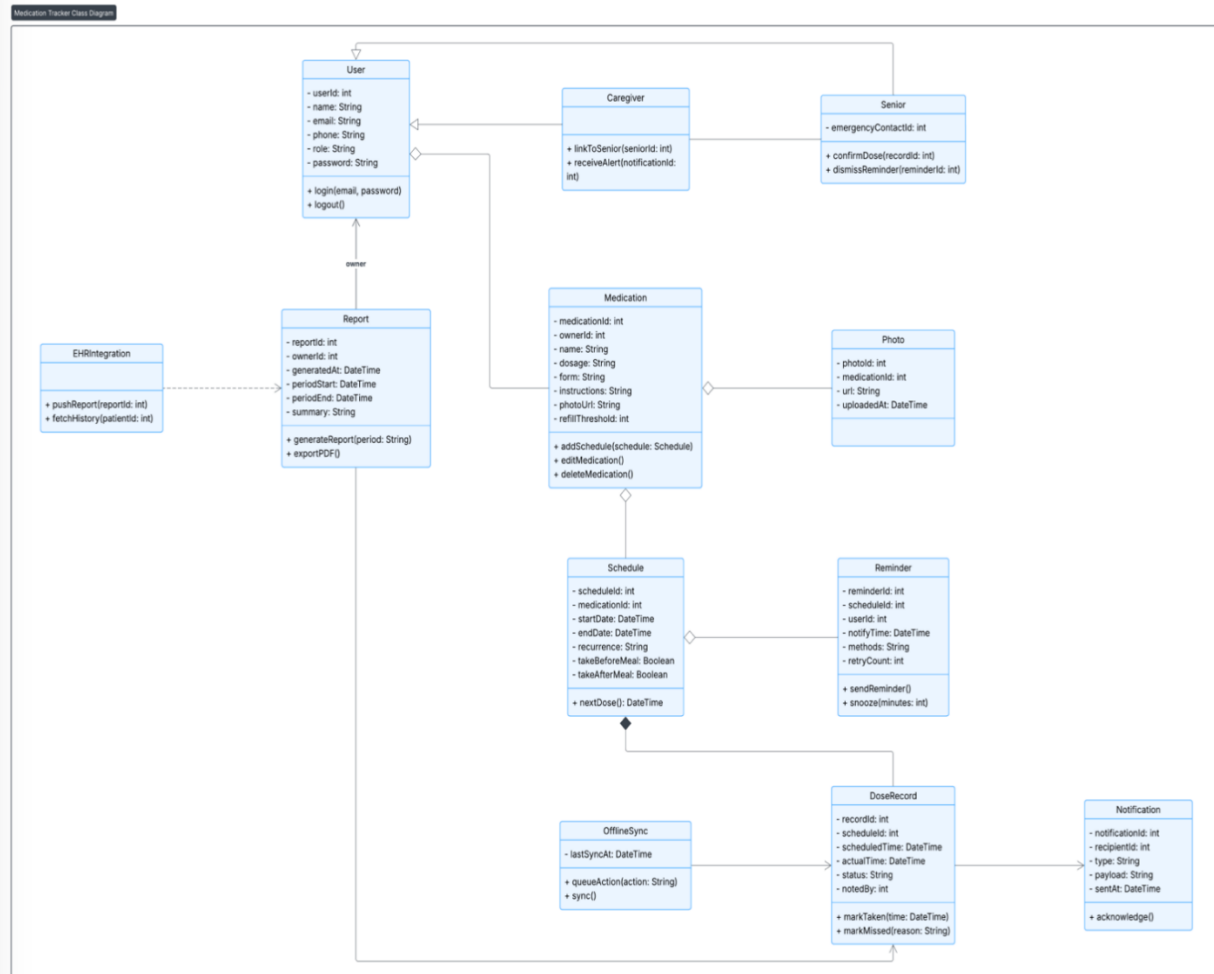


Architecture summary

- ✓ **Client** side (mobile/web app) manage **UI**, local storage and synchronization process.
- ✓ **Server** side consist of three layers: **API** – Application services (Business layer) – Data access.
- ✓ Our architecture provides separation between presentation and logic and data handling.
- ✓ Using client server improves security, scalability and maintainability, also using this architecture supports offline functionality by using local storage caching.

Class Diagram Description

The **Class Diagram** represents the *Medication Tracker System* structure and how its main components interact.



- The main class is **User**, which has two subtypes:
 - Senior**: The person taking the medication.
 - Caregiver**: A family member or helper who monitors the senior.
- Each **Senior** can have multiple **Medications**, and each medication can include multiple **Schedules** defining when and how often it should be taken (e.g., daily or after lunch).
- For every scheduled dose, a **DoseRecord** is created to track whether the medication was taken or missed.
- Reminders** notify the senior at the correct time, and if a dose is missed, a **Notification** is sent to the caregiver.

- **Photos** can be attached to medications to help seniors identify their pills easily.
 - **A Report** can be generated to show the full medication history and shared with doctors.
 - The system includes two important components:
 - **OfflineSync** stores user actions locally and syncs later.
 - **EHRIntegration** enables future connection with healthcare systems.
-

How OfflineSync Works

The **OfflineSync** feature ensures the system functions normally even without internet access by temporarily saving user actions locally and synchronizing them later.

Steps:

User performs actions offline marking doses, editing reminders, creating schedules.

System detects no internet the app avoids sending failed requests.

Action is saved locally in a queue using SQLite or local storage.

User continues normally reminders and logs still function offline.

Connectivity checks system periodically tests for internet.

Automatic synchronization when online, all offline actions are uploaded and confirmed.

Error handling failed sync attempts are retried until success.

Conclusion

The Medication Tracker System successfully addresses the real-world problem of seniors forgetting or mismanaging their medications by providing a reliable, accessible, and user-friendly digital solution. Through features such as reminder scheduling, dose confirmation, caregiver alerts, and offline synchronization, the system enhances medication adherence and communication between patients, caregivers, and healthcare providers.

The project demonstrates the application of system analysis and design principles using UML diagrams, functional and non-functional requirements, and a well-structured architecture based on a client-server and layered model. The implementation of

OfflineSync further improves usability and reliability by ensuring uninterrupted functionality even without an internet connection.

Overall, the Medication Tracker contributes to improving seniors' quality of life, increasing safety, and reducing caregiver stress—reflecting the potential of technology to promote healthier, more independent living for older adults.